

CSA0927 - PROGRAMMING IN JAVA

NAME : G.RAJU

REG NO : 192411065

FACULTY : S.PUSPHALATHA

1.

The screenshot shows a Java IDE with a 'PROGRAM EDITOR' tab. On the left, a 'QUESTIONS' sidebar shows '5/8 completed' and a 'Logger' question. The main editor displays the following code:

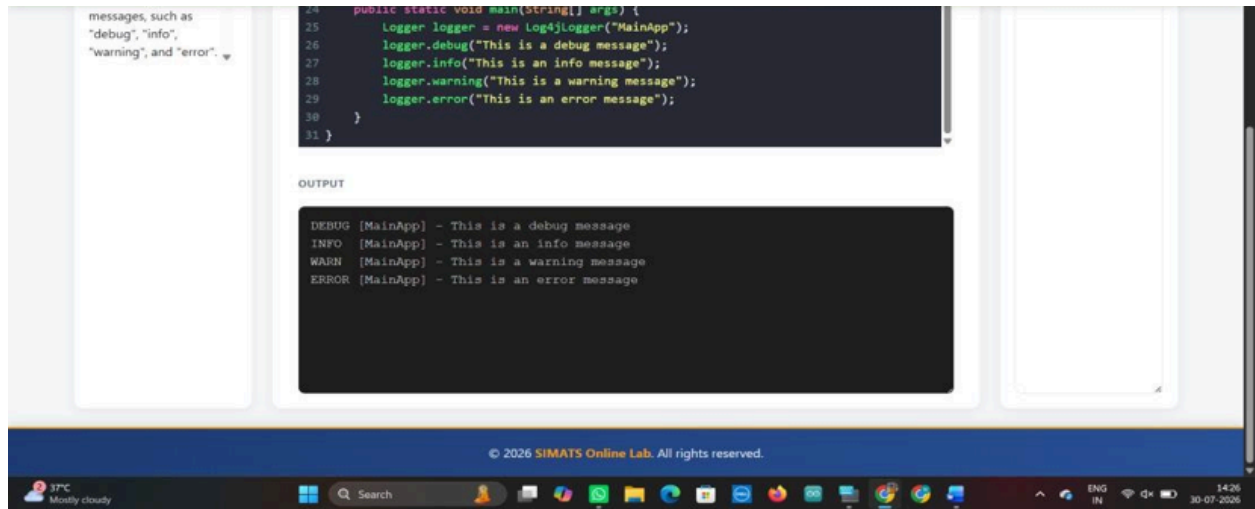
```
1 interface Logger {
2     void debug(String message);
3     void info(String message);
4     void warning(String message);
5     void error(String message);
6 }
7 class Log4jLogger implements Logger {
8     private String className;
9     public Log4jLogger(String className) {
10         this.className = className;
11     }
12     public void debug(String message) {
13         System.out.println("DEBUG [" + className + "] - " + message);
14     }
15     public void info(String message) {
16         System.out.println("INFO [" + className + "] - " + message);
17     }
18     public void warning(String message) {
19         System.out.println("WARN [" + className + "] - " + message);
20     }
21     public void error(String message) {
```

The right sidebar has an 'INPUT' section with the text 'Your INPUT goes here!'. The bottom status bar shows '37°C Mostly cloudy', a search bar, and system icons for language (ENG IN), battery, and time (14:25 30-07-2026).

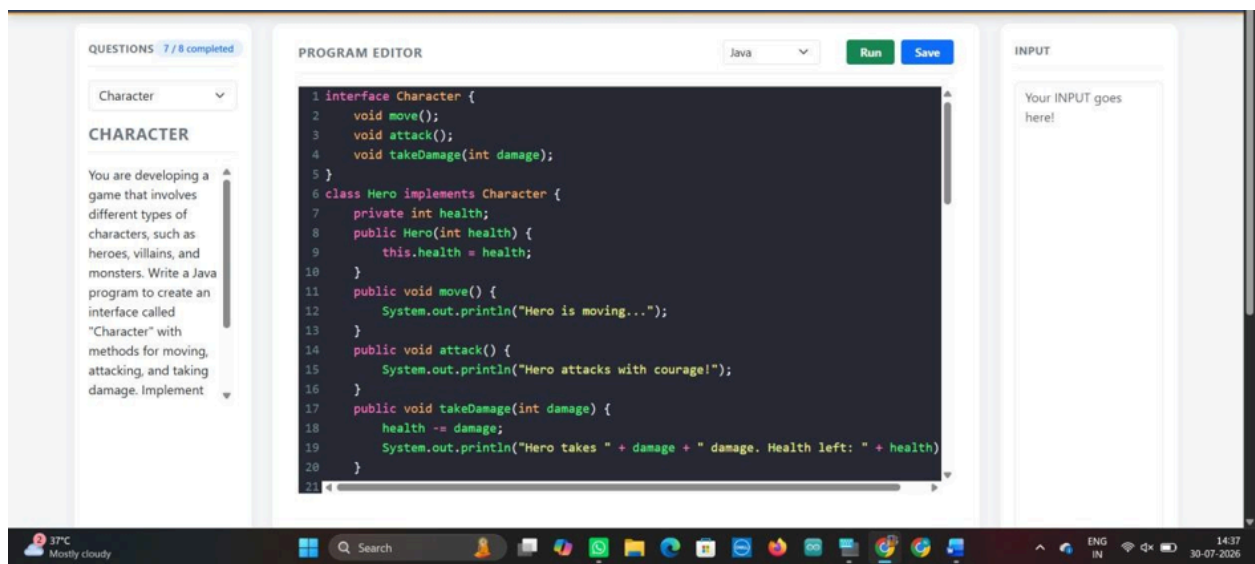
This screenshot shows the same Java IDE with the code completed. The 'PROGRAM EDITOR' tab now includes the following code:

```
22     System.out.println("ERROR [" + className + "] - " + message);
23 }
24 public static void main(String[] args) {
25     Logger logger = new Log4jLogger("MainApp");
26     logger.debug("This is a debug message");
27     logger.info("This is an info message");
28     logger.warning("This is a warning message");
29     logger.error("This is an error message");
30 }
31 }
```

The 'OUTPUT' section at the bottom now displays the result of the first log statement: 'DEBUG [MainApp] - This is a debug message'. The right sidebar and bottom status bar remain the same as in the previous screenshot.



2.



QUESTIONS 7/8 completed

Character

CHARACTER

You are developing a game that involves different types of characters, such as heroes, villains, and monsters. Write a Java program to create an interface called "Character" with methods for moving, attacking, and taking damage. Implement

PROGRAM EDITOR

Java Run Save

```
21 }
22 class Villain implements Character {
23     private int health;
24     public Villain(int health) {
25         this.health = health;
26     }
27     public void move() {
28         System.out.println("Villain is creeping...");
29     }
30     public void attack() {
31         System.out.println("Villain attacks with evil plans!");
32     }
33     public void takeDamage(int damage) {
34         health -= damage;
35         System.out.println("Villain takes " + damage + " damage. Health left: " + health);
36     }
37 }
38 class Monster implements Character {
39     private int health;
40     public Monster(int health) {
```

INPUT

Your INPUT goes here!

37°C Mostly cloudy

Search

ENG IN 14:38 30-07-2026

3.

Sort

SORT

Write a Java program to create a list of integers and sort them in ascending order using the Collections framework.

PROGRAM EDITOR

```
1 import java.util.*;
2 public class Main {
3     public static void main(String[] args) {
4         List<Integer> numbers = new ArrayList<>();
5         numbers.add(5);
6         numbers.add(2);
7         numbers.add(9);
8         numbers.add(1);
9         numbers.add(7);
10        Collections.sort(numbers);
11        System.out.println("Sorted List: " + numbers);
12    }
13 }
```

OUTPUT

Sorted List: [1, 2, 5, 7, 9]

INPUT

Your INPUT goes here!

37°C Mostly cloudy

Search

ENG IN 14:39 30-07-2026

4.

HashSet

HASHSET

Write a Java program to create a HashSet of strings and remove a specific element from it.

PROGRAM EDITOR

```
1 import java.util.*;
2 public class HashSet {
3     public static void main(String[] args) {
4         HashSet<String> set = new HashSet<>();
5         set.add("Apple");
6         set.add("Banana");
7         set.add("Mango");
8         set.add("Grapes");
9         System.out.println("Before removal: " + set);
10        String removalElement = "Banana";
11        boolean removed = set.remove(removalElement);
12        System.out.println("Removed \"" + removalElement + "\"? " + removed);
13        System.out.println("After removal: " + set);
14    }
15 }
```

OUTPUT

Security Error: Disallowed code pattern detected.

INPUT

Your INPUT goes here!

37°C Mostly cloudy

Search

ENG IN 14:27 30-07-2026

5.

TreeMap

TREEMAP

Write a Java program to create a TreeMap of strings and integers and iterate over its entries.

```
1 import java.util.*;
2 public class TREEMAP{
3     public static void main(String[] args){
4         TreeMap<Integer,String>map=new TreeMap<>();
5         map.put(100,"john");
6         map.put(101,"sri");
7         map.put(102,"harsha");
8         System.out.println(map);
9     }
10 }
```

OUTPUT

{100=john, 101=sri, 102=harsha}

Your INPUT goes here!

37°C Mostly cloudy

Search

ENG IN 14:30 30-07-2026

6.

PriorityQueue

PRIORITYQUEUE

Write a Java program to create a PriorityQueue of integers and add elements to it.

```
1 import java.util.PriorityQueue;
2 public class Main {
3     public static void main(String[] args) {
4         PriorityQueue<Integer> pq = new PriorityQueue<>();
5         pq.add(10);
6         pq.add(5);
7         pq.add(20);
8         pq.add(1);
9         pq.add(15);
10        System.out.println("PriorityQueue elements in order:");
11        while (!pq.isEmpty()) {
12            System.out.print(pq.poll() + " ");
13        }
14    }
15 }
```

OUTPUT

PriorityQueue elements in order:
1 5 10 15 20

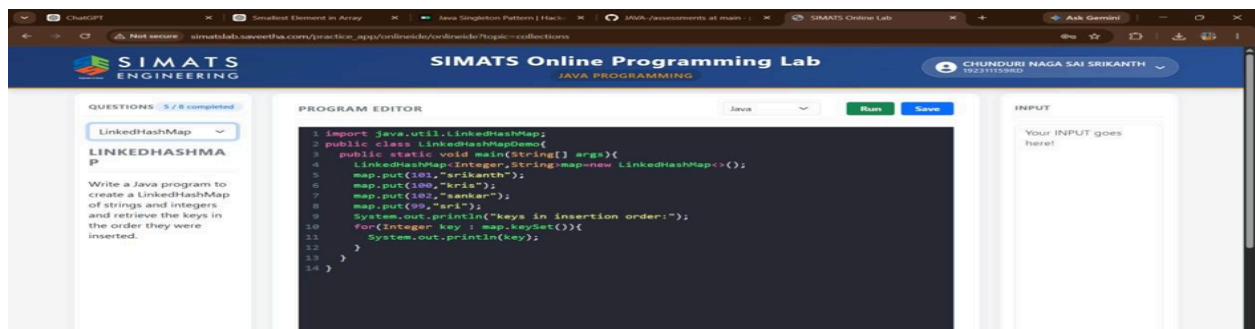
Your INPUT goes here!

37°C Mostly cloudy

Search

ENG IN 14:33 30-07-2026

7.



8.

